

Vectorizing the Trie: Efficient Constrained Decoding for LLM-based Generative Retrieval on Accelerators

Zhengyang Su*
Google

Mountain View, CA, USA
susteven@google.com

Isay Katsman*[†]
Yale University

New Haven, CT, USA
isay.katsman@yale.edu

Yueqi Wang*
Google

New York, NY, USA
yueqi.wang@google.com

Ruining He
Google

Mountain View, CA, USA
ruininghe@google.com

Lukasz Heldt
Google

Mountain View, CA, USA
heldt@google.com

Raghunandan
Keshavan

Google
Mountain View, CA, USA
hkraghunandan@google.com

Shao-Chuan Wang
Google

Mountain View, CA, USA
scwang@google.com

Xinyang Yi
Google

Mountain View, CA, USA
xinyang@google.com

Mingyan Gao
Google

Mountain View, CA, USA
mingyan@google.com

Onkar Dalal
Google

Mountain View, CA, USA
onkardalal@google.com

Lichan Hong
Google

Mountain View, CA, USA
lichan@google.com

Ed H. Chi
Google

Mountain View, CA, USA
edchi@google.com

Ningren Han

Google
Mountain View, CA, USA
peterhan@google.com

Abstract

Generative retrieval has emerged as a powerful paradigm for LLM-based recommendation. However, industrial recommender systems often benefit from restricting the output space to a constrained subset of items based on business logic (e.g. enforcing content freshness or product category), which standard autoregressive decoding cannot natively support. Moreover, existing constrained decoding methods that make use of prefix trees (Tries) incur severe latency penalties on hardware accelerators (TPUs/GPUs). In this work, we introduce **STATIC** (Sparse Transition Matrix-Accelerated Trie Index for Constrained Decoding), an efficient and scalable constrained decoding technique designed specifically for high-throughput LLM-based generative retrieval on TPUs/GPUs. By flattening the prefix tree into a static Compressed Sparse Row (CSR) matrix, we transform irregular tree traversals into fully vectorized sparse matrix operations, unlocking massive efficiency gains on hardware accelerators.

We deploy STATIC on a large-scale industrial video recommendation platform serving billions of users. STATIC produces significant product metric impact with minimal latency overhead (0.033 ms per step and 0.25% of inference time), achieving a 948 $\times$ speedup over a CPU trie implementation and a 47–1033 $\times$ speedup over a

hardware-accelerated binary-search baseline. Furthermore, the runtime overhead of STATIC remains extremely low across a wide range of practical configurations. To the best of our knowledge, STATIC enables the first production-scale deployment of strictly constrained generative retrieval. In addition, evaluation on academic benchmarks demonstrates that STATIC can considerably improve cold-start performance for generative retrieval. Our code is available at <https://github.com/youtube/static-constraint-decoding>.

CCS Concepts

• Computing methodologies $\rightarrow$ Machine learning approaches.

Keywords

Generative Retrieval, Constrained Decoding, Semantic ID, Trie, Sparse Matrix, TPU, GPU, Recommender Systems, Large Language Models.

ACM Reference Format:

Zhengyang Su, Isay Katsman, Yueqi Wang, Ruining He, Lukasz Heldt, Raghunandan Keshavan, Shao-Chuan Wang, Xinyang Yi, Mingyan Gao, Onkar Dalal, Lichan Hong, Ed H. Chi, and Ningren Han. 2026. Vectorizing the Trie: Efficient Constrained Decoding for LLM-based Generative Retrieval on Accelerators. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3770855.3818506>

*These authors contributed equally to this work.

[†]This work was performed while the author was participating in the Google Student Researcher Program at Google.



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '26, Jeju Island, Republic of Korea*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818506>

1 Introduction

Recommender systems have long served as the core discovery engines powering massive online platforms like YouTube and Amazon [3, 21]. While neural network techniques continue to improve

these engines, this domain is undergoing a paradigm shift whereby embedding-based neural retrieval models that recommend candidates through approximate nearest neighbor search such as ScaNN [7, 30] are being superseded with LLM-based generative retrieval [5, 8, 27, 29, 34, 36]. By representing each item as a sequence of discrete tokens (called Semantic IDs or SIDs) and training Transformers [32] to autoregressively decode the Semantic ID tokens of the target items [27], generative retrieval bypasses key limitations of traditional dual (user, item)-encoders [33] to capture deeper semantic relationships. Additionally, it obviates the need for an external nearest-neighbor indexing infrastructure [27] and allows one to adapt pre-trained LLMs for recommendation tasks [8, 22, 36].

However, generative retrieval models as they were originally designed lack a critical feature: control over the generated output space, as required to enforce business logic across a variety of industry applications. The absence of this feature has not stopped these next generation recommendation models from becoming widespread in large-scale industry settings [5, 8, 37, 38], but more fine-grained output space control would enable practitioners to restrict the output space based on business logic, such as enforcing freshness constraints (e.g. “uploaded within the last day”), locality (e.g. regional recommendations), product categories (e.g. “recommend summer clothes”), or inventory availability (e.g. “in stock” only). This makes it possible to train a single large model and deploy it across multiple use cases.

In a traditional recommendation setting, business logic is enforced by filtering items post-retrieval or constraining the search space a priori (e.g. by running a restricted ScaNN search [7]). If a system requires items to be “in stock” or “uploaded within 7 days,” a boolean filter simply drops invalid candidates from the retrieved set or the ScaNN index. In generative retrieval, however, the “retriever” is a probabilistic autoregressive model that synthesizes Semantic IDs [27]. Without intervention, an LLM will confidently generate SIDs for items that are out-of-stock, stale, or legally restricted. Relying on post-generation filtering is computationally wasteful; the model may spend its entire inference budget generating invalid items, resulting in zero valid recommendations.

To address this issue, constraints must be enforced *during* the LLM decoding process. The standard algorithmic solution is to implement constrained decoding with a prefix tree (trie), which masks invalid tokens at each step [35]. While conceptually sound, we observe that naive pointer-chasing trie-based constraints are fundamentally hostile to hardware accelerators for two reasons:

- (1) **Memory Latency** Pointer-based structures result in non-contiguous, random memory access patterns. This prevents memory coalescing, failing to utilize the High-Bandwidth Memory (HBM) burst capabilities of modern TPUs/GPUs [14]. Furthermore, these irregular patterns nullify hardware prefetchers designed for linear streaming, leading to severe memory-bound latency and stalled execution units.
- (2) **Compilation Incompatibility** Modern accelerator architectures (e.g. Google’s XLA-reliant [28] TPUs) require static computation graphs for ML compilation [19]. Naive trie implementations rely on data-dependent control flow and irregular linked structures, which are fundamentally incompatible

with this paradigm. While accelerators support dynamic indexing into fixed-size buffers (e.g. vectorized gather), the irregular memory access and recursive branching of pointer-chasing preclude end-to-end ML compilation.

In our preliminary experiments, a CPU-offloaded trie implementation increased inference time by 2×, rendering it unusable for our target per-decoding-step latency of ≤ 10 ms. More sophisticated existing approaches utilizing binary search also exhibited significant latency penalties [35]. While alternative methods like Finite State Transducers (FSTs) are standard in natural language processing, they suffer from state explosion when applied to production-scale SID vocabularies (e.g. on the order of millions of items) and lack native TPU support [18].

In this work, we introduce **STATIC** (Sparse Transition Matrix-Accelerated Trie Index for Constrained Decoding), a framework that recasts constrained decoding from a graph traversal problem into a series of vectorized sparse matrix operations. STATIC is designed to resolve the aforementioned issues and enable extremely fast and scalable constrained decoding for generative retrieval. Notably, our I/O complexity¹ is $O(1)$ with respect to constraint set size, in contrast with the logarithmic scaling from existing binary search-based methods.

Our contributions are as follows:

- (i) We propose a method to flatten prefix tree-specified constraints into static Compressed Sparse Row (CSR) matrices, enabling $O(1)$ memory access overhead via coalesced reads for fast decoding constraint extraction.
- (ii) We design a branch-free decoding algorithm using dynamic slicing and mask arithmetic. This makes constrained decoding fully accelerator-native and eliminates host-device round-trips, thereby enabling massive efficiency gain.
- (iii) We deploy this system on an ultra-large-scale video recommendation platform (YouTube) for multiple product use cases. We present one specific setting in which we constrain an generative retrieval LLM to a pre-specified vocabulary of 20 million fresh items, improving key online metrics with minimal latency overhead compared to baseline methods.
- (iv) We evaluate the scalability of STATIC and find that its latency remains extremely low across a wide range of constraint set and Semantic ID vocabulary sizes.
- (v) We demonstrate how one can use constrained decoding to improve cold-start recommendation performance in a principled setting using Amazon Reviews [9] datasets.

2 Related Work

In the recommender systems field, the shift from embedding-based retrieval [7] to generative retrieval [27] has necessitated the design of novel Transformer decoding strategies. Our work lies at the intersection of large-scale generative recommendation [27], constrained token generation, and hardware-aware system optimization.

¹Unlike traditional time or space complexity, I/O complexity [11] on a TPU evaluates algorithmic efficiency by counting the number of costly data block transfers between off-chip High Bandwidth Memory (HBM) and on-chip SRAM.

2.1 Generative Retrieval and Semantic Indexing

Generative retrieval (GR) represents a paradigm shift wherein the item corpus is internalized within the model parameters, replacing the traditional large-scale embedding and nearest neighbor search pipeline with a direct sequence generation task [31].

Recent approaches in this area, specifically TIGER [27], SEATER [29], and LIGER [34], utilize Semantic IDs—hierarchical, discrete token-based item representations, where semantically similar items are optimized to share the same prefix. These generative retrieval techniques produce improved results and have been deployed in multiple online recommendation platforms, as seen in PLUM [8] and OneRec [5, 22, 37, 38]. However, a critical limitation of these autoregressive retrieval models is the “validity gap.” Unlike natural language generation, where the output space is open-ended, retrieval requires the generated string to map perfectly to a valid item index. As noted by Rajput et al. [27], without constraints, the model effectively “hallucinates” non-existent inventory, leading to retrieval failures. This issue is especially pertinent in industry settings where practitioners deploy the same pre-trained model across multiple product environments, each having its own valid corpus, with some requiring candidates to fall within a certain regional locality or meet certain “freshness” requirements. While previous work focuses on improving recall metrics [5, 8, 15, 29, 37], it largely ignores the latency costs of enforcing validity constraints in a high-throughput production environment; we explicitly address this in our paper.

Additionally, existing generative retrieval papers [34] have noted that generative retrieval models struggle with cold-start item recommendation: the setting in which a model must recommend items it has never before seen during training. We demonstrate in Section 6 that by implementing constrained decoding with a cold-start item set, one can improve cold-start performance significantly.

2.2 Constrained Decoding in NLP

Constrained decoding (CD) has a rich history in NLP. For example, NeuroLogic [23] uses search-based heuristics to satisfy complex logical predicates, and SynchroMesh [24] enforces Context-Free Grammars (CFGs) to ensure syntactic correctness in code generation. An approach related to our use case is that of Finite State Transducers (FSTs), which have long been the standard for constrained speech recognition [18]. However, FSTs suffer from state explosion when applied to the unstructured, high-cardinality vocabularies found in recommendation systems (often 10^7+ items). As observed by Koo et al. [18], while FSTs are powerful, their irregularity makes them difficult to parallelize on TPUs/GPUs compared to dense matrix operation-based methods.

2.3 Hardware-Aware Inference and Acceleration

The “Memory Wall” remains the primary bottleneck for Large Language Model inference. Because autoregressive decoding is memory-bandwidth bound, performance is dictated by how fast data moves to the chip’s arithmetic units [4].

Pointer Chasing vs. Coalesced Access. Standard trie implementations used in various constrained decoding approaches rely on pointer chasing [20]. On modern accelerators (TPUs/GPUs), pointer chasing induces random memory access patterns that lead

to uncoalesced loads and cache thrashing. This is antithetical to the design of accelerators, which thrive on static, contiguous memory access (as seen in optimizations like FlashAttention [4]).

Accelerator-Compatible Decoding. The recent paper DISC-PPV [35] represents the state-of-the-art in adapting constrained decoding to hardware accelerators. The authors implement an on-chip validity check using a flattened sorted array and parallelized binary search, a method they call Parallel Prefix-Verification (PPV). While PPV eliminates the CPU-GPU communication overhead, its reliance on binary search introduces an I/O complexity scaling factor of $O(\log |C|)$ with respect to the constrained item set size $|C|$. In ultra-large vocabularies (e.g. 20 million items), even logarithmic scaling becomes an I/O bottleneck. Our introduced STATIC approach advances this line of research by replacing the $O(\log |C|)$ dependent block transfers of PPV with $O(1)$ vectorized sparse matrix operations, fetching all required working sets in a single coalesced phase.

2.4 Linearized Tree Structures

The concept of flattening tree structures into array-based representations dates back to the Double-Array Trie [13], originally designed to improve memory efficiency for string matching. Similarly, the GraphBLAS standard [17] posits that graph algorithms can be expressed as linear algebra operations over sparse semirings.

While these concepts are established in theoretical computer science, their application to LLM constrained decoding is novel. We bridge the gap between these classical data structures and modern deep learning compilers (XLA/Inductor). By reformulating trie traversal via sparse matrix operations, we enable the use of high-performance, ML-compiler-accelerated kernels that maintain the parallelism required for real-time recommendation at scale.

3 Background

In this section, we provide the background necessary to understand our paper and define the technical terminology central to generative retrieval and constrained decoding.

3.1 Generative Retrieval and Semantic IDs

Generative retrieval is a recommendation paradigm wherein a model directly predicts target candidate identifiers token-by-token rather than through nearest-neighbor search in an embedding space. Central to this approach is the concept of the Semantic ID: a discrete, token-based identifier that captures the semantic properties of an item.

In the TIGER framework [27], Semantic IDs are created using a Residual-Quantized Variational AutoEncoder (RQ-VAE). An RQ-VAE first encodes item features into a latent vector $\mathbf{z}$, which is then iteratively quantized across L levels. The first-level residual is defined as: $\mathbf{r}_1 := \mathbf{z}$. At each level d , the residual $\mathbf{r}_d$ is identified with the nearest embedding $\mathbf{e}_{y_d}$ in a level-specific codebook $\mathcal{E}_d$. The index y_d becomes the d -th codeword in the Semantic ID. The next-level residual is computed as: $\mathbf{r}_{d+1} := \mathbf{r}_d - \mathbf{e}_{y_d}$.

This recursive approach allows semantically similar items to share prefix tokens, enabling the model to effectively generalize across the item space. The final resulting Semantic ID is simply the tuple comprised of the embedding indices, i.e. $(y_1, \dots, y_L)$.

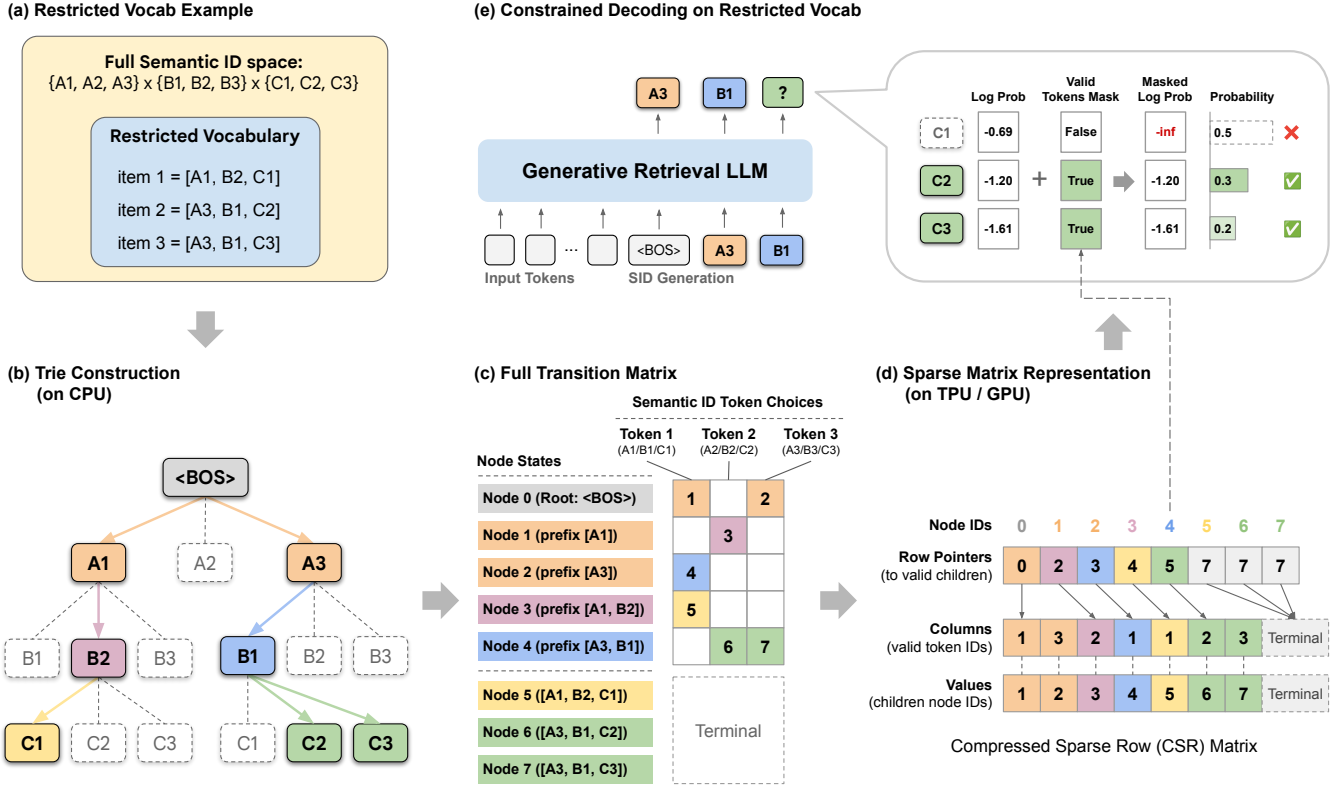


Figure 1: This figure showcases the full STATIC pipeline. Figures 1a and 1b present the prefix tree construction for the case $\mathcal{V} = \{1, 2, 3\}$, $L = 3$, and restricted vocabulary $C = \{(1, 2, 1), (3, 1, 2), (3, 1, 3)\}$. We prepend the letters A, B, C to the semantic tokens to denote first, second, and third levels, respectively. The corresponding transition matrix is then shown in Figure 1c, with explicit labels and color-coding. Figure 1d presents the sparse matrix (CSR) representation of the full transition matrix. Figure 1e shows how the sparse matrix is applied to constrain the vocabulary at decoding time.

3.2 Autoregressive Decoding via Beam Search

During inference, the retrieval model autoregressively decodes the token sequences of Semantic IDs. To generate target items we typically employ beam search [6] over the Semantic ID space (although our approach is also compatible with other decoding methods). Let $\mathcal{V}$ be the vocabulary of semantic tokens and let L be the fixed length of a Semantic ID. At each step t , the algorithm maintains a beam search state S_t which tracks the M most probable sequences across a batch of size B . We formally define S_t as a structure with the following members and accessors:

- $S_t.\text{tokens} \in \mathcal{V}^{B \times M \times t}$: The token buffer containing the decoded prefixes for each beam.
- $S_t.\text{scores} \in \mathbb{R}^{B \times M}$: The cumulative log-probability scores for the sequences.

At step t , the model generates logits $\mathbf{L}_t \in \mathbb{R}^{B \times M \times |\mathcal{V}|}$ which are normalized into log-probabilities before selecting the next set of candidates. The cumulative log-probability score is updated by adding the model's predicted log-probabilities for the next token to the current prefix scores. The top M candidates with the highest cumulative scores are retained, while others are pruned.

3.3 Prefix Trees and Constrained Decoding

Standard generative models can “hallucinate” identifiers that either (1) do not map to valid items in the corpus or (2) do not map to desirable items (e.g. too old, in the context of freshness). To prevent this, constrained decoding restricts the output to a restricted vocabulary set C containing only desirable Semantic IDs.

Because Semantic IDs are structured as sequences, the restricted vocabulary inherently forms a prefix tree (i.e. trie). Constraints are enforced at each step t , that is, if appending the current token to the prefix results in a invalid path in the trie, the path is pruned. In practice, the log-probabilities of invalid tokens are set to $-\infty$, ensuring that beam search only selects valid sequences. The challenge addressed in this work is that naive pointer-chasing implementations of trie constraints incur severe latency penalties on hardware accelerators like TPUs/GPUs, making them infeasible in practice.

4 Methodology

We propose an accelerator-native constrained decoding technique designed for LLM-based generative retrieval. Our approach replaces dynamic pointer-chasing CPU-based prefix tree traversals with a

static sparse matrix lookup, enabling purely on-device execution (TPU/GPU) compatible with XLA/Inductor compilation.

4.1 Problem Formulation

Recall that $\mathcal{V}$ represents the set of semantic tokens (i.e. indices) in our context, and L , the fixed length of a Semantic ID. Note that the space of all Semantic IDs is simply the product space $\mathcal{V}^L$. Although the vocabulary of indices is the same across all levels, the codewords are different, with a different codebook $\mathcal{E}_d$ trained for each level.

The retrieval model generates a sequence (i.e. a Semantic ID) $y = (y_1, \dots, y_L)$ auto-regressively, where $y_i \in \mathcal{V}$. In standard beam search, the validity of a sequence is determined solely by the model's learned probability distribution P_θ . However, in constrained decoding we require the generated output to belong to a strictly defined subset $C \subset \mathcal{V}^L$ (e.g. a "Freshness" corpus or "Inventory" list). An example of a restricted subset is given in Figure 1a.

We formally define constraint functions $F_t : \mathcal{V}^{t-1} \times \mathcal{V} \rightarrow \{0, 1\}$ for $1 \leq t < L$ by: $F_t(y_{<t}, y_t) = \mathbb{I}(\exists c \in C \text{ s.t. } (y_{<t}, y_t) \sqsubseteq c)$, where $\mathbb{I}(\cdot)$ is the indicator function and $\sqsubseteq$ denotes prefix inclusion. $F_t(y_{<t}, y_t)$ is an indicator function that returns 1 if and only if appending y_t to the prefix $y_{<t}$ results in a valid prefix in C . Our goal is to enforce this constraint only during inference, such that $P(y_t | y_{<t}) = 0$ if $F_t(y_{<t}, y_t) = 0$, with minimal latency overhead.

4.2 Sparse Transition Matrix (STM)-based Trie Conversion

Standard implementations of prefix trees rely on pointer-chasing operations that have a huge CPU callback bottleneck and are poorly supported on accelerators. To resolve this, we flatten the prefix tree into a Compressed Sparse Row (CSR) transition matrix.

Let the number of total prefix nodes in the trie be S . We map every unique prefix node in the tree to a state integer $s \in [S]$. We then define a sparse matrix $T \in \mathbb{Z}^{S \times |\mathcal{V}|}$ where:

$$T_{s,v} = \begin{cases} s_{\text{next}} & s \xrightarrow{v} s_{\text{next}} \text{ exists} \\ 0 & \text{otherwise} \end{cases}$$

Conceptually, 0 denotes a terminal state where no more transitions are possible (think of it as a "sink"). To better illustrate the idea, we give the conceptual transition matrix for the restricted vocabulary from Figure 1a in Figure 1c (corresponding to the trie in Figure 1b).

4.2.1 Matrix Construction. The construction of T is performed offline. Given the high sparsity of the vocabulary constraints (often $\leq 0.01\%$ of valid paths), the CSR representation is highly memory efficient. In our case, we fill the three parts of the CSR representation as follows:

- **Row Pointers (P):** Stores indices for the allowed transitions for current state s , i.e. these are effectively node pointers.
- **Column Indices (C):** Stores the valid token IDs (v) that trigger transitions.
- **Values Array (V):** Stores the target state node IDs (s_{next}).

This static structure allows us to utilize hardware-optimized sparse matrix operations rather than dynamic control flow. To illustrate, for the transition matrix in Figure 1c we provide the CSR arrays in Figure 1d. Notice, for example, to extract the transitions from Node 4 in Figure 1c, we would first extract $\text{row_start} = P[4] = 5$ and

$\text{row_end} = P[4 + 1] = 7$. Then we form the slices:

$$C[\text{row_start}:\text{row_end}] = C[5:7] = [2, 3]$$

$$V[\text{row_start}:\text{row_end}] = V[5:7] = [6, 7]$$

to observe that taking "Token 2" from Node 4 leads to Node 6, and taking "Token 3" from Node 4 leads to Node 7. One can confirm that this is indeed correct by noting the transitions present in Figure 1c.

4.3 Accelerator-Native Decoding Algorithm

To achieve high throughput on hardware accelerators, we reformulate the validity check as a vectorized lookup. Critically, we maintain a transition state vector $\mathbf{n}_t \in \mathbb{Z}^{B \times M}$ that tracks the current node index in the prefix tree for each beam. Additionally, we employ the following optimization. For a number of layers $0 \leq d < L$, we

maintain a dense tensor mask $\mathbf{D} \in \mathbb{R}^{|\mathcal{V}| \times \dots \times |\mathcal{V}|}$ that indicates precisely which d -length prefixes exist in the restricted vocabulary C . Lookups in this tensor are extremely fast and the construction of $\mathbf{D}$ is a one-time fixed cost; however, $\mathbf{D}$ becomes prohibitively expensive to construct for $d \geq 3$ in most cases due to the fact that $|\mathcal{V}|^d$ grows exponentially in d . Hence, we have $d \leq 2$ for most real-world use cases and rely on the sparse CSR transition matrix for deeper layers. The full sequential constrained decoding process is given in Algorithm 1. Our approach relies on several hardware-accelerated operations:

Algorithm 1: Hardware-Accelerated Constrained Decoding Step

Input: Logits $L_t \in \mathbb{R}^{B \times M \times |\mathcal{V}|}$, Beam State S_{t-1} , Step t

Input: Dense Count d , Dense Restrict Tensor $\mathbf{D} \in \mathbb{R}^{|\mathcal{V}| \times \dots \times |\mathcal{V}|}$ d times
Input: Constraints: Trans. Matrix T , Node Indices $\mathbf{n}_{t-1} \in \mathbb{Z}^{B \times M}$
Output: Updated State S_t , New Node Indices $\mathbf{n}_t$

- 1 **Phase 1: Log-Space Projection**
 - 2 $P_t \leftarrow \text{LogSoftmax}(L_t)$ // Convert logits to log-probs
 - 3 **Phase 2: Constraint Masking**
 - 4 **if** $t - 1 < d$ **then**
 - 5 $(\mathbf{n}_{\text{next}}, \mathbf{m}) \leftarrow \text{DenseLookup}(\mathbf{n}_{t-1}, \mathbf{D}, t - 1)$
 - 6 **else**
 - 7 $(\mathbf{n}_{\text{next}}, \mathbf{m}) \leftarrow \text{VNTK}(\mathbf{n}_{t-1}, T, t - 1)$ // Execute Algorithm 2
 - 8 **end**
 - 9 $P'_t \leftarrow \text{Where}(\mathbf{m}, P_t, -\infty)$ // Mask invalid transitions
 - 10 **Phase 3: Beam Search Optimization**
 - 11 $(S_{\text{best}}, I_{\text{best}}) \leftarrow \text{BeamSearch}(P'_t, S_{t-1}.\text{scores}, M)$
 - 12 **Phase 4: State Update**
 - 13 $S_t.\text{tokens} \leftarrow \text{Gather}(S_{t-1}.\text{tokens}, \text{indices} = I_{\text{best}})$
 - 14 $S_t.\text{scores} \leftarrow S_{\text{best}}$
 - 15 $\mathbf{n}_t \leftarrow \text{Gather}(\mathbf{n}_{\text{next}}, \text{indices} = I_{\text{best}})$ // Advance constraint state
 - 16 **return** $S_t, \mathbf{n}_t$
-

- **LogSoftmax**($\mathbf{x}$): Converts raw logits into normalized log-probabilities according to the following transformation: $\text{LogSoftmax}(\mathbf{x})_i = x_i - \log \sum_{j=1}^{|\mathcal{V}|} \exp(x_j)$.
- **DenseLookup**($\mathbf{n}_{t-1}, \mathbf{D}, t-1$): This primitive performs a high-performance lookup on the dense tensor mask $\mathbf{D}$ to identify valid semantic extensions for each beam.
- **VNTK**($\mathbf{n}_{t-1}, \mathbf{T}, t-1$): The *Vectorized Node Transition Kernel* (Algorithm 2). It performs a high-performance lookup on the sparse transition matrix $\mathbf{T}$ to identify valid semantic extensions for each beam. It encapsulates the speculative slicing and projection logic required to return the next-node IDs $\mathbf{n}_{next}$ and the dense log probability mask $\mathbf{m}$ in a single vectorized pass. See Section 4.4 for more detail.
- **Where**(mask, $\mathbf{x}$, value): This conditional operator enforces validity constraints in log-probability space. It prunes invalid paths by setting their scores to $-\infty$ where the mask $\mathbf{m}$ is false.
- **BeamSearch**($\mathbf{P}'_t, \mathbf{S}_{scores}, M$): A selection framework that identifies M most probable valid sequences across the batch. It computes cumulative scores by combining the new masked log-probabilities $\mathbf{P}'_t$ with existing prefix scores $\mathbf{S}_{scores}$ and retains the top candidates for the next step.
- **Gather**($\mathbf{x}$, indices): A vectorized selection operator used in the state update phase (Phase 4). It extracts the specific tokens, scores, and constraint states from the candidate pool that correspond to the top-performing beams ($\mathbf{I}_{best}$).

4.4 Implementation and Hardware Optimization

While the theoretical formulation of STATIC guarantees correctness, achieving negligible latency overhead on modern accelerators requires rigorous alignment with the underlying hardware characteristics. In this section, we detail the vectorized kernel design that makes our approach portable across both TPUs and GPUs.

The core engineering challenge in constrained decoding is handling the dynamic nature of the prefix tree (where nodes possess a varying number of children) within the static execution model of hardware accelerators. Standard pointer-chasing traversals are inefficient: on TPUs (XLA), dynamic branching triggers costly graph recompilation or forces the use of serial ‘while’ loops that prevent hardware pipelining; on GPUs, processing beams with mismatched child counts leads to GPU warp divergence, where threads must wait for the longest path to finish, destroying SIMT parallelism.

To overcome these bottlenecks, we implement a branch-free transition kernel (Algorithm 2). For efficient slicing, it becomes critical to introduce some new definitions. Consider the prefix tree corresponding to a sparse transition matrix $\mathbf{T}$. For a node n in the tree at level ℓ , let the number of children be b_n^ℓ . Given that the tree has L levels, let the set of all nodes at level $\ell \in [L]$ be N_ℓ and define the vector $\mathbf{B} \in \mathbb{R}^L$ such that $\mathbf{B}_\ell = \max_{n \in N_\ell} b_n^\ell$. That is, $\mathbf{B}$ is the level-indexed vector of max branch factors exhibited by the tree. Note that computing this vector is a one-time fixed cost per transition matrix $\mathbf{T}$. Our algorithm takes this vector as an input. Moreover, Algorithm 2 relies on the following vectorized primitive operations:

- **DynamicSlice**($\mathbf{M}$, start, len): Extracts a contiguous sub-tensor of fixed length $len = \mathbf{B}_t$ from the data matrix $\mathbf{M}$ starting at a

dynamic offset (*start*). Recall decoding step t is synonymous with level ℓ in the underlying prefix tree.

- **Range**(c): Generates a constant sequence vector $[0, \dots, c-1]$ used for index-based masking and sanitization of the speculatively sliced children.
- **Scatter**(indices, values): A projection operator that converts the sparse list of valid tokens into a dense boolean mask $\mathbf{m}$ of size $|\mathcal{V}|$. This mask is applied directly to the model’s log-probabilities in the main decoding step to enforce the prefix-tree constraints.

As shown in Phase 2, the kernel always slices a fixed number of entries $\mathbf{B}_t$ for any given level t regardless of the actual child count. If a node has fewer than $\mathbf{B}_t$ children, we use a **Range**-generated vector to create a boolean mask $\mathbf{m}_{valid}$ and the **Where** operator to sanitize the result. This ensures that the arithmetic units of the GPU/TPU remain fully saturated and that the entire decoding step remains a single, static computation graph. We discuss additional relevant hardware considerations of our method in Appendix A. We also benchmark our method for high maximum branch factors in Appendix E and demonstrate linear scaling on this axis. Importantly, we note that for sufficiently large $|\mathcal{V}|$ and practical ranges of $|\mathcal{C}|$, the max branch factor will actually remain quite low for later layers, since the number of children $|\mathcal{V}|^\ell$ grows exponentially with level ℓ and quickly exceeds the constraint set size $|\mathcal{C}|$.

5 Large-scale Deployment on YouTube

By testing STATIC at YouTube scale, we seek to evaluate the STATIC algorithm along three dimensions critical for industrial deployment:

- (1) **System Efficiency**: We measure latency and throughput on TPU accelerators compared to baselines.

Algorithm 2: Vectorized Node Transition Kernel (VNTK)

Input: Curr. Node n_{curr} , Trans. Matrix $\mathbf{T}$ (Row Pointers $\mathbf{P}$, Columns $\mathbf{C}$, Values $\mathbf{V}$), Step t

Input: (Constant) Token Cardinality $|\mathcal{V}|$, Max Branch Factors $\mathbf{B}$

Output: Next Nodes $\mathbf{n}_{next}$, Logit Mask $\mathbf{m}$

1 Phase 1: Boundary Lookup

2 $idx_{start} \leftarrow \mathbf{P}[n_{curr}]$

3 $N_{child} \leftarrow \mathbf{P}[n_{curr} + 1] - idx_{start}$ // Compute # of children

4 Phase 2: Speculative Slicing

// Slice fixed size $\mathbf{B}_t$ regardless of actual child count

5 $\mathbf{d}_{col} \leftarrow \text{DynamicSlice}(\mathbf{C}, \text{start} = idx_{start}, \text{len} = \mathbf{B}_t)$

6 $\mathbf{d}_{val} \leftarrow \text{DynamicSlice}(\mathbf{V}, \text{start} = idx_{start}, \text{len} = \mathbf{B}_t)$

7 Phase 3: Sanitization (Branch-Free)

8 $\mathbf{J} \leftarrow \text{Range}(\mathbf{B}_t)$

// Generate vector $[0, \dots, \mathbf{B}_t - 1]$

9 $\mathbf{m}_{valid} \leftarrow (\mathbf{J} < N_{child})$

// Identify valid slots

10 $\mathbf{t}_{valid} \leftarrow \text{Where}(\mathbf{m}_{valid}, \mathbf{d}_{col}, \{\})$

11 $\mathbf{n}_{next} \leftarrow \text{Where}(\mathbf{m}_{valid}, \mathbf{d}_{val}, 0)$

12 Phase 4: Projection

// Create dense boolean mask for Softmax

13 $\mathbf{m} \leftarrow \text{Scatter}(\text{indices} = \mathbf{t}_{valid}, \text{values} = \mathbf{m}_{valid})$

14 **return** $\mathbf{n}_{next}, \mathbf{m}$

- (2) **Scalability:** We analyze the storage footprint and runtime as both constraint set size and vocabulary size increase.
- (3) **Online Product Impact:** We quantify the improvement of recommendation quality and user satisfaction from applying constrained decoding in a live production environment.

5.1 Experimental Setup

We conduct our evaluation on a large-scale video recommendation corpus from YouTube. The videos are tokenized into Semantic IDs (SIDs) [27] with $L = 8$ discrete tokens and token cardinality $|\mathcal{V}| = 2048$. The total recommendable corpus is on the order of 100 million to 1 billion items, and the unconstrained model can recommend any item from this set. We fix $d = 2$ as an input to Algorithm 1 and maintain the dense tensor mask $\mathbf{D} \in \mathbb{R}^{2048 \times 2048}$ for early lookups; we use the CSR sparse matrix representation for later levels.

The model is a Gemini-based generative retrieval model similar to PLUM [8], served with a batch size of 2 (per chip) and a beam size of $M = 70$. The model is based on a non-Mixture-of-Experts (MoE) architecture with 3 billion dense parameters. All benchmark experiments are conducted on Google TPU v6e accelerators.

5.2 System Efficiency Analysis

We limit our attention to a constrained vocabulary (target set) consisting of a “fresh video” subset containing approximately 20 million high-quality items uploaded in the last 7 days (relative to the experiment date). This setup represents a restrictive environment for constrained decoding: the valid subset is sparse, yet large enough (20 million items) to make naive enumeration infeasible.

We compare our STATIC approach against several distinct implementations² commonly found in academic literature and production systems:

- (1) **Unconstrained:** Standard beam search with no validity checks. This represents the lower bound on latency.
- (2) **CPU Trie:** A standard prefix tree stored in CPU memory using JAX’s PyTree data structure (Python dictionary). At every decoding step, the TPU halts, sends partial beams to the CPU, waits for the validity mask, and resumes.

²We did not compare against Finite State Transducers (FSTs) for fundamental reasons we highlight in Appendix C.

Table 1: Latency Overhead per Decoding Step on a Large-scale Video Recommendation Corpus with 20 Million Constrained Vocabulary Items. STATIC (Ours) achieves minimal latency overhead relative to baselines.

Method	Latency (ms)	% of Inference
Unconstrained	+0.0	–
PPV Exact [35]	+34.1	260%
CPU Trie	+31.3	239%
Hash Bitmap	+12.3	94.0%
PPV Approximate [35]	+1.56	11.9%
STATIC (Ours)	+0.033	0.25%

- (3) **PPV Exact [35]:** Following Ye et al. [35], we store valid SIDs in a sorted flat array on TPU and perform binary search to verify every candidate extension. In the original paper, the authors consider only the top 50 logits of the vocabulary per decoding step. This is suboptimal for beam search with larger fan-out since constrained set items frequently fall outside of the top 50 logits. Thus, in this baseline, we consider the PPV method using all 2048 logits. This yields an exact solution but is significantly slower than PPV Approximate below, due to PPV’s linear scaling with the number of logits.
- (4) **PPV Approximate [35]:** Just as above, we perform binary search on a sorted flat SID array for every candidate extension; however, as in the original paper, this baseline only verifies the top 50 logits per decoding step. This gives only approximate solutions for the reasons mentioned above, and is not directly comparable to STATIC.
- (5) **Hash Bitmap:** A Bloom-filter [1] style approach where valid prefixes are hashed. This method introduces false positives (with a non-negligible 2.1% false positive rate in our Table 1 evaluation), so it is not directly comparable to STATIC, but useful as a reference.

STATIC is a lossless implementation optimization of the trie-based constrained decoding algorithm, so we focus on measuring the latency of the constraint enforcement logic per decoding step. Our results are summarized in Table 1, where we give means over 100 trials. Our STATIC approach achieves a latency overhead of +0.033 ms per decoding step, only taking 0.25% of the inference time for a 3 billion parameter model. In contrast, the “CPU Trie” method incurs a massive penalty (31.3 ms, 948× higher than STATIC) due to the PCIe transfer overhead and synchronization locks between the TPU and CPU. Crucially, we outperform the “PPV Exact” baseline by a factor of 1033× (34.1 ms vs. 0.033 ms) and even the “PPV Approximate” baseline by factor of 47× (1.56 ms vs. 0.033 ms). While PPV’s binary search is technically “on-device” and vectorized, it suffers from logarithmic scaling of I/O complexity with respect to the constraint set size $|\mathcal{C}|$ (on top of linear scaling with respect to number of logits considered), whereas STATIC reduces the I/O complexity to $O(1)$ using the single-pass VNTK kernel (Algorithm 2).

5.3 Scalability

While a naive boolean mask of the entire Semantic ID space would be petabytes in size, our STATIC approach requires only approximately 90 MB of memory for every 1 million restricted vocabulary items in this setting. That is, for our tested restricted vocabulary of 20 million items, we expect a maximal HBM usage of ≈ 1.8 GB, which we can further tighten to ≈ 1.5 GB via the analysis in Appendix B. In practice, the required memory is usually $\leq 75\%$ of this upper limit. We elaborate on the derivations and why this is the case in Appendix B. The crucial takeaway is that the memory usage is tied directly to the storage of the dense tensor mask $\mathbf{D}$ and the sparse CSR matrix, which is tied to the number of nodes in the underlying trie. Given a vocabulary size $|\mathcal{V}|$, Semantic ID length L , dense layer count d , number of constraints $|\mathcal{C}|$, storage cost per node of K_1 bytes, and storage cost per dense state of K_2 bytes, the

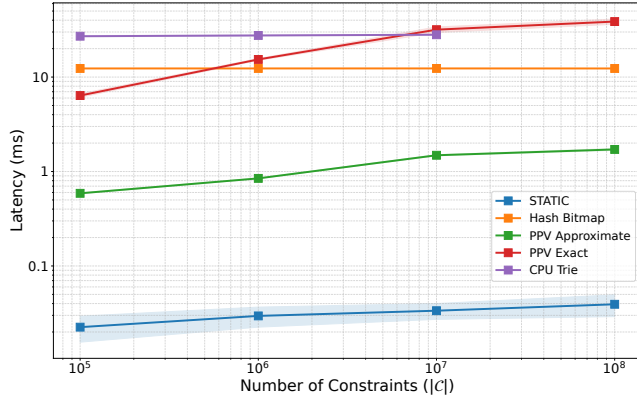


Figure 2: Scaling of different constraint decoding methods relative to constraint set size (log-log scale), fixing $|\mathcal{V}| = 2048$. STATIC considerably outperforms existing methods.

upper bound on memory usage (in bytes) is given by:

$$U_{\max}(K_1, K_2, |\mathcal{V}|, |C|, L, d) = \left(\frac{1}{8} + K_2\right) |\mathcal{V}|^d + K_1 \cdot \sum_{\ell=d+1}^L \min(|\mathcal{V}|^\ell, |C|)$$

Additionally, STATIC’s time complexity scales logarithmically with respect to constraint set size and maintains low latency relative to all measured baselines across a wide range of $|C|$. A direct comparison with the Table 1 baselines is given in Figure 2, where we generate constraint sets uniformly at random, fixing $|\mathcal{V}| = 2048$, $L = 8$ and vary only $|C|$ from 10^5 to 10^8 . The means over 100 trials are plotted and the shaded region indicates one standard deviation above and below the mean³.

Furthermore, STATIC exhibits almost constant latency with respect to the SID vocabulary size $|\mathcal{V}|$, as shown in Figure 3, where we generate constraint sets uniformly at random just as above, fixing $|C| = 10^7$, $L = 8$, and vary only $|\mathcal{V}|$ from 256 to 32k. In theory, the pure sparse matrix look-ups scale with maximum branch factor (as shown in Algorithm 2), whose average (under mild assumptions) scales linearly with $|\mathcal{V}|$. However, since we employ a dense mask with $O(1)$ look-up for the highly saturated first $d = 2$ layers, the sparse look-ups happen for later layers with small constant maximum branch factors. As such, STATIC exhibits near constant scaling with $|\mathcal{V}|$. In almost all production systems [8, 37] as of 2025, $|\mathcal{V}| \leq 8192$, meaning STATIC exhibits consistently low latency across all current real-world settings. Exact numbers for Figures 2 and 3 are given in Appendix D.

5.4 Online A/B Testing

We deployed a STATIC-enabled PLUM-based generative retrieval model across multiple use cases in the YouTube ecosystem. In one setting, we used it as an additional candidate source to a short-form video product’s “Home Feed” with a “Last 7 Days” freshness constraint, and conducted A/B experiments to measure the impact. The results are shown in Table 2.

As expected, STATIC achieved 100% compliance with the specified constraint. A standard generative retrieval model without this

³The $|C| = 10^8$ data point for CPU Trie is missing due to an Out-Of-Memory error.

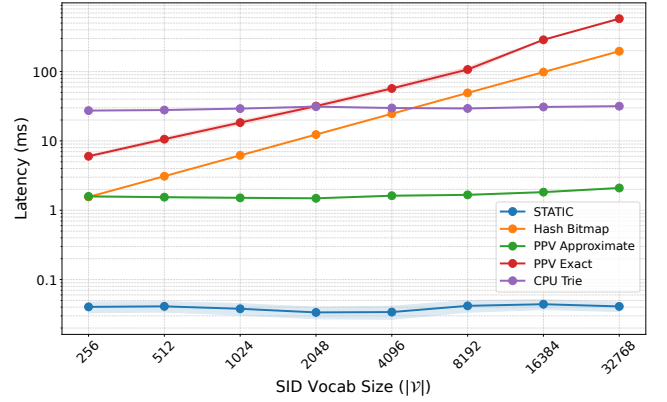


Figure 3: Scaling of different constraint decoding methods relative to SID vocabulary size $|\mathcal{V}|$ (log-log scale) at $|C| = 10^7$. STATIC compares favorably for all tested SID vocab sizes.

Table 2: Online A/B Test Results for “Home Feed” Impact with the “Last 7 Days” Freshness Constraint.

Metric	Impact	95% Confidence Interval
7 Day Fresh Views	+5.1%	[5.0%, 5.2%]
3 Day Fresh Views	+2.9%	[2.8%, 3.0%]
Click-Through Rate	+0.15%	[0.01%, 0.29%]
Strategic User Segment Satisfaction	+0.15%	[0.03%, 0.27%]

constraint would frequently generate SIDs for older videos, whereas the STATIC model strictly produced valid 7-day fresh videos.

We observed a significant increase in the consumption of fresh content: as given in Table 2, we note a +5.1% increase in 7-day fresh video views and a +2.9% increase in 3-day fresh video views.

Moreover, we observed a positive impact on user experience metrics: the click-through rate (CTR) increased by +0.15% and user satisfaction of a strategic user segment increased by +0.15%. This is a significant improvement and shows the positive impact of recommending more high-quality fresh videos.

6 Cold-Start Retrieval on Amazon Reviews Datasets

A long-standing drawback of generative retrieval is its inability to generalize to cold-start items [34]. Here, we show how one can use constrained decoding on the cold-start item set to considerably improve cold-start performance. We use the Amazon Reviews [9] datasets to showcase a viable set-up and the associated benefits.

6.1 Experimental Setup

The Amazon items are tokenized into Semantic IDs (SIDs) by training separate RQ-VAE [27] models to quantize the items of each Amazon Reviews subdataset separately, with $L = 4$ discrete tokens and token cardinality $|\mathcal{V}| = 256$. Each subdataset’s valid item corpus consists of anywhere from 10 to 20 thousand items, and the unconstrained model can recommend any item from this corpus.

We train a separate generative retrieval model on the sequences from each Amazon subdataset based on the Gemma [16] architecture with 1 billion dense parameters, served with a batch size of 16 and a beam size of $M = 20$. All experiments are conducted on a Google TPU v6e.

6.2 Cold-Start Results

We perform cold-start experiments in the following way. For a given Amazon item, we define its “age” by the age of its oldest review. Thereafter, for each Amazon subdataset, we isolate the newest 2% (and separately, 5%) of the items into a cold-start set. We filter out all item sequences that do not contain any items from this set, and train the models on these “no cold-start train sequences.” We evaluate the models on the “cold-start test sequence” set which contains sequences that have cold-start items as targets. We tested the following approaches for retrieval:

- (1) **Unconstrained**: Beam search with no validity checks.
- (2) **Constrained Random Guessing**: A baseline that guesses items uniformly at random from the constraint set.
- (3) **Dense Retrieval [12]**: A separately trained two-tower model whose framework follows a variant of dense retrieval as presented by Hou et al. [12]. We give the architectural and training details for this model in Appendix F.
- (4) **STATIC (Ours)**: Our proposed method, where the model decodes from the Transformer trained in (1) while restricting to the cold-start item set at every step.

Table 3 shows Recall@1 metrics for the “cold-start test sequences.” In both the 2% and 5% cold-start settings, STATIC improves over the unconstrained, constrained random guessing, and dense retrieval approaches considerably, showing that one can attain nontrivial cold-start performance with generative retrieval using constrained decoding *alone*. Additional context about the evaluation protocol is given in Appendix F.

Table 3: Comparison of Unconstrained and Constrained Decoding Performance on Amazon Cold-start Items. All values given are Recall@1 percentages.

Subdataset	Method	2% Cold-start	5% Cold-start
Beauty	Unconstrained	0.00%	0.00%
	Const. Random	0.42%	0.17%
	Dense Retrieval	0.88%	0.63%
	STATIC	4.29%	1.60%
Sports & Outdoors	Unconstrained	0.00%	0.00%
	Const. Random	0.27%	0.11%
	Dense Retrieval	0.85%	0.36%
	STATIC	1.24%	1.17%
Toys & Games	Unconstrained	0.00%	0.00%
	Const. Random	0.42%	0.17%
	Dense Retrieval	0.49%	0.44%
	STATIC	4.39%	2.25%

7 Conclusion

Generative retrieval holds the promise of leveraging LLMs for next-generation recommendations in myriad real-world settings, but its

adoption in industry has been hindered by the lack of a controllable output space. In this work, we bridged the gap between prefix tree-based constraints and vector-based hardware (TPUs/GPUs). Regarding trie-constrained decoding, the authors of Ye et al. [35] claim: “whether an efficient GPU trie algorithm exists remains to be [seen].” This is a hypothesis that we answer in the affirmative with the contribution in our paper. The STATIC framework transforms pointer-chasing trie lookups into vectorized sparse matrix operations, achieving a 47–1033 $\times$ speedup over alternative on-device methods. We demonstrated the effectiveness of this approach in production environments serving billions of users, showing that strict constraints can be enforced at scale without compromising serving latency. Moreover, we demonstrated the viability of improving cold-start performance through constrained-decoding alone on the Amazon Reviews [9] datasets.

Future Work. Currently the sparse transition matrix construction is an offline process. An important future extension is to develop dynamic sparse updates, which would allow for real-time inventory changes without full model recompilation.

References

- [1] Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13 (1970), 422–426. <https://api.semanticscholar.org/CorpusID:7931252>
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nécule, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python–NumPy programs*. <http://github.com/google/jax>
- [3] Paul Covington, Jay K. Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. *Proceedings of the 10th ACM Conference on Recommender Systems* (2016). <https://api.semanticscholar.org/CorpusID:207240067>
- [4] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *ArXiv abs/2205.14135* (2022). <https://api.semanticscholar.org/CorpusID:249151871>
- [5] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment. *ArXiv abs/2502.18965* (2025). <https://api.semanticscholar.org/CorpusID:276617997>
- [6] Markus Freitag and Yaser Al-Onaizan. 2017. Beam Search Strategies for Neural Machine Translation. In *NMT@ACL*. <https://api.semanticscholar.org/CorpusID:2229477>
- [7] Ruiqi Guo, Philip Sun, Erik M. Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2019. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *International Conference on Machine Learning*. <https://api.semanticscholar.org/CorpusID:218614141>
- [8] Ruining He, Lukasz Heldt, Lichan Hong, Raghu Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Y. Tsai, Yueqi Wang, Shao-Chuan Wang, Xinyang Yi, Lexi Baugher, Baykal Cakici, Ed Huai hsin Chi, Cristos Goodrow, Ningren Han, He Ma, Romer Rosales, Abby Van Soest, Devansh Tandon, Su-Lin Wu, Weilong Yang, and Yilin Zheng. 2025. PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations. *ArXiv abs/2510.07784* (2025). <https://api.semanticscholar.org/CorpusID:281950827>
- [9] Ruining He and Julian McAuley. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. *Proceedings of the 25th International Conference on World Wide Web* (2016). <https://api.semanticscholar.org/CorpusID:1964279>
- [10] Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. 2024. *Flax: A neural network library and ecosystem for JAX*. <http://github.com/google/flax>
- [11] Jia-Wei Hong and H. T. Kung. 1981. I/O complexity: The red-blue pebble game. In *Symposium on the Theory of Computing*. <https://api.semanticscholar.org/CorpusID:8410593>
- [12] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (2022). <https://api.semanticscholar.org/CorpusID:249625869>
- [13] Jun ichi Aoe. 1989. An Efficient Digital Search Algorithm by Using a Double-Array Structure. *IEEE Trans. Software Eng.* 15 (1989), 1066–1077. <https://api.semanticscholar.org/CorpusID:249625869>

- semanticscholar.org/CorpusID:267894119
- [14] Norman P. Jouppi, Cliff Young, Nishant Patil, David A. Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre Luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Taraneh Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Daniel Killebrew, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. 2017. In-datacenter performance analysis of a tensor processing unit. *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)* (2017), 1–12. <https://api.semanticscholar.org/CorpusID:4202768>
 - [15] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvesh Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*. 6420–6425. doi:10.1145/3746252.3761612
 - [16] Gemma Team Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ram'e, Morgane Riviere, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean-Bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gael Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Róbert Iştván Busa-Fekete, Alex Feng, Naveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Iordan Brusicovskiy, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesh Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, Andr'as Gyorgy, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Boxi Wu, Bobak Shahriari, Bryce Pettrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, Cj Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szepkator, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, J. Michael Wieting, Jonathan Lai, Jordi Orbay, Joe Fernandez, Joshua Newlan, Junsong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stańczyk, Pouya Dehghani Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Ardeshtir Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Pöder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vladimir Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab S. Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeffrey Dean, Demis Hassabis, Koray Kavukcuoglu, Clément Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and L'eonard Hussenot. 2025. Gemma 3 Technical Report. *ArXiv abs/2503.19786* (2025). <https://api.semanticscholar.org/CorpusID:277313563>
 - [17] Jeremy Kepner, Peter Aaltonen, David A. Bader, Aydın Buluç, Franz Franchetti, John R. Gilbert, Dylan Hutchison, Manoj Kumar, Andrew Lumsdaine, Henning Meyerhenke, Scott McMillan, Carl Yang, John Douglas Owens, Marcin Zalewski, Timothy G. Mattson, and José E. Moreira. 2016. Mathematical foundations of the GraphBLAS. *2016 IEEE High Performance Extreme Computing Conference (HPEC)* (2016), 1–9. <https://api.semanticscholar.org/CorpusID:3654505>
 - [18] Terry Koo, Frederick Liu, and Luheng He. 2024. Automata-based constraints for language model decoding. *ArXiv abs/2407.08103* (2024). <https://api.semanticscholar.org/CorpusID:271097802>
 - [19] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, and Depei Qian. 2020. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Transactions on Parallel and Distributed Systems* 32 (2020), 708–727. <https://api.semanticscholar.org/CorpusID:211069666>
 - [20] Hao Liao, Jiwei Zhang, Jianxun Lian, Wensheng Lu, Mingqi Wu, Shuo Wang, Yong Zhang, Yitian Huang, Mingyang Zhou, and Rui Mao. 2025. Eliminating Out-of-Domain Recommendations in LLM-based Recommender Systems: A Unified View. <https://api.semanticscholar.org/CorpusID:278339501>
 - [21] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Comput.* 7 (2003), 76–80. <https://api.semanticscholar.org/CorpusID:263872610>
 - [22] Zhanyun Liu, Shiyao Wang, Xing-Yao Wang, Rong-Qing Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, Yifei Hu, Qigen Hu, Xinchun Luo, Lejian Ren, Zixing Zhang, Qianqian Wang, Kuo Cai, Yun-Jie Wu, Hongtao Cheng, Zexuan Cheng, Lu Ren, Huanjie Wang, Yi Su, Ruiming Tang, Kun Gai, and Guorui Zhou. 2025. OneRec-Think: In-Text Reasoning for Generative Recommendation. *ArXiv abs/2510.11639* (2025). <https://api.semanticscholar.org/CorpusID:282058058>
 - [23] Ximing Lu, Sean Welleck, Peter West, Liwei Jiang, Jungo Kasai, Daniel Khashabi, Ronan Le Bras, Lianhui Qin, Youngjae Yu, Rowan Zellers, Noah A. Smith, and Yejin Choi. 2021. NeuroLogic A'esque Decoding: Constrained Text Generation with Lookahead Heuristics. In *North American Chapter of the Association for Computational Linguistics*. <https://api.semanticscholar.org/CorpusID:245218671>
 - [24] Gabriel Poesia, Oleksandr Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable code generation from pre-trained language models. *ArXiv abs/2201.11227* (2022). <https://api.semanticscholar.org/CorpusID:246294475>
 - [25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *J. Mach. Learn. Res.* 21 (2019), 140:1–140:67. <https://api.semanticscholar.org/CorpusID:204838007>
 - [26] Samyann Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2019. ZeRO: Memory optimizations Toward Training Trillion Parameter Models. *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis* (2019), 1–16. <https://api.semanticscholar.org/CorpusID:269617042>
 - [27] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Hieu Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. *ArXiv abs/2305.05065* (2023). <https://api.semanticscholar.org/CorpusID:258564854>
 - [28] Amit Sabne. 2020. Xla: Compiling machine learning for peak performance.
 - [29] Zihua Si, ZhongXiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, and Jun Xu. 2023. Generative Retrieval with Semantic Tree-Structured Identifiers and Contrastive Learning. *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region* (2023). <https://api.semanticscholar.org/CorpusID:262466132>
 - [30] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2024. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. *ArXiv abs/2404.00774* (2024). <https://api.semanticscholar.org/CorpusID:268030651>
 - [31] Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. 2022. Transformer Memory as a Differentiable Search Index. *ArXiv abs/2202.06991* (2022). <https://api.semanticscholar.org/CorpusID:246863488>
 - [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:13756489>
 - [33] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. 2026. On the Theoretical Limitations of Embedding-Based Retrieval. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=k9CzlvzfaA>
 - [34] Liu Yang, Fabian Paischer, Kaveh Hassani, Jiacheng Li, Shuai Shao, Zhang Gabriel Li, Yun He, Xue Feng, Nima Noorshams, Sem Park, Bo Long, Robert Nowak, Xiaoli Gao, and Hamid Eghbalzadeh. 2024. Unifying Generative and Dense Retrieval for Sequential Recommendation. *Trans. Mach. Learn. Res.* 2025 (2024). <https://api.semanticscholar.org/CorpusID:274423313>
 - [35] Haotian Ye, Himanshu Jain, Chong You, Ananda Theertha Suresh, Haowei Lin, James Zou, and Felix X. Yu. 2025. Efficient and Asymptotically Unbiased Constrained Decoding for Large Language Models. *ArXiv abs/2504.09135* (2025). <https://api.semanticscholar.org/CorpusID:277781124>
 - [36] Guorui Zhou, Honghui Bao, Jiaming Huang, Jiaxin Deng, Jinghao Zhang, Junda She, Kuo Cai, Lejian Ren, Lu Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rongzhou Zhang, Ruiming Tang, Shiyao Wang, Wuchao Li, Xiangyu Wu, Xinchun Luo, Xingmei Wang, Yifei Hu, Yunfan Liu, Zhanyun Liu, Zhiyang Zhang, Zixing Zhang, Bo Chen, Bin Wen, Chaoyi Ma, Chengru Song, Chenglong Chu, Defu Lian, Fan Yang, Feng Jiang, Hongtao Cheng, Huanjie Wang, Kun Gai, Pengfei Zheng, Qiang Wang, Rui Huang, Siyang Mao, Tingting Gao, Wei Yuan, Yan Wang,

- Yang Zhou, Yi Su, Zexuan Cheng, Zhixin Ling, and Ziming Li. 2026. OpenOneRec Technical Report. arXiv:2512.24762 [cs.LG]. <https://arxiv.org/abs/2512.24762>
- [37] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, Weifeng Ding, Wuchao Li, Xinchun Luo, Xing-Yao Wang, Zexuan Cheng, Zixing Zhang, Bin Zhang, Bo-Wen Wang, Chao Ma, Cheng bin Song, Chenhui Wang, Di Wang, Dongxue Meng, Fan Yang, Fang-Peng Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Hengrui Hu, Hezheng Lin, Hongtao Cheng, Hong Cao, Huanjie Wang, Jiaming Huang, Jiapeng Chen, Jiaqian Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Liao Yu, Qiang Wang, Qidong Zhou, Shengzhe Wang, Shi He, Shuang Yang, Shu-Jun Yang, Sui Huang, Tao Wu, TIAN-CHEN He, Tingting Gao, Wei Yuan, Xiaoyan Liang, Xiao-Xue Xu, Xugang Liu, Yan Wang, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yun-Hong Wu, Yunfeng Zhao, and Zhanyun Liu. 2025. OneRec Technical Report. *ArXiv abs/2506.13695* (2025). <https://api.semanticscholar.org/CorpusID:279410085>
- [38] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, Pengfei Zheng, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Ruiming Tang, Shiyao Wang, Shu-Jun Yang, Tao Wu, Wuchao Li, Xin-Jing LuO, Xing-Yao Wang, Yi Su, Yun-Jie Wu, Zexuan Cheng, Zhanyun Liu, Zixing Zhang, Bin Zhang, Bo-Long Wang, Chao Ma, Cheng bin Song, Chenhui Wang, Chenglong Chu, Di Wang, Dongxue Meng, Dunju Zang, Fan Yang, Fang-Peng Zhang, Fengqing Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Honghui Bao, Hong Cao, Jiaming Huang, Jiapeng Chen, Jiaqian Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Qiang Wang, Qidong Zhou, Rong-Qing Zhang, Shengzhe Wang, Shi He, Shuang Yang, Si-Yuan Mao, Sui Huang, TIAN-CHEN He, Tingting Gao, Wei Yuan, Xiaofeng Liang, Xiao-Xue Xu, Xugang Liu, Yan Wang, Yang Zhou, Yi Wang, Yiwu Liu, Yuexin Song, Yufei Zhang, Yunfeng Zhao, Zhixin Ling, and Zi-Yue Li. 2025. OneRec-V2 Technical Report. *ArXiv abs/2508.20900* (2025). <https://api.semanticscholar.org/CorpusID:280949827>

Appendix

A Hardware Optimization Discussions

We shall now discuss some of the additional hardware-related details of our constrained decoding implementation.

A.1 JIT Compilation and Data Structures

The XLA compilers for TPUs require static tensor shapes at compile time to generate optimized fusion kernels. However, standard trie traversals are inherently dynamic: node A may have 2 children, while node B may have 1000. In a CPU environment, one would simply allocate dynamic vectors. But in an XLA context, chasing pointers with dynamic vectors would trigger compilation errors.

Our Vectorized Node Transition Kernel (VNTK) (Algorithm 2) addresses this using static gather operations. Instead of processing a variable number of children, we process precisely B_ℓ , where this value is the maximum branch factor at level ℓ . We then use a vectorized **Gather** operation (e.g., `jnp.take` with `mode='fill'`) to fetch exactly B_ℓ potential transitions for every beam in parallel.

To handle nodes with $N_{child} < B_\ell$ children, we compute a validity mask on the fly ($m_{valid} = \text{Range}(B_\ell) < N_{child}$) to zero out invalid entries in the score calculation. This transformation converts a dynamic control flow problem (“loop N times”) into a static data flow problem (“gather fixed B_ℓ elements and mask”). This ensures the entire decoding step remains a single, static XLA graph, enabling full loop unrolling and pipelining on TPU systolic arrays.

A.1.1 Stacked CSR Layout. To further minimize memory access latency of VNTK, we employ a stacked CSR memory layout. In a standard CSR format, column indices and data values are stored in separate arrays, typically requiring two memory fetches per transition. In STATIC, we stack these into a single $(N_{edges}, 2)$ tensor. This allows coalesced reads to retrieve both the candidate token ID and the pointer to the next node in a single memory transaction, effectively halving the number of random memory accesses.

A.1.2 Dense Mask Optimization. While sparse traversal is efficient for deep layers of the trie, the initial layers exhibit very high branching factors. To address this, for the first d decoding steps (where $0 \leq d < L$), we bypass the VNTK and utilize a pre-computed dense boolean mask tensor to represent all valid prefixes of length d . This hybrid approach allows us to trade a small amount of static memory for a significant throughput gain in the most computationally expensive layers of the retrieval process. In our experiments in Section 5, we set $d = 2$ to densify the first two layers of the trie.

A.1.3 Variable Length SIDs. Additionally, we want to highlight that while our evaluation focused on fixed-length SIDs with a fixed-size discrete codebook, the CSR-based transition matrix and the VNTK kernel are inherently compatible with variable-length sequences with variable-size codebooks. This is because a transition can be drawn to a null node at any level of the prefix tree and because the branch factor does not have to be fixed so long as we have an index assigned to every node, which is a requirement of the CSR format.

A.2 Cross-Platform Portability (JAX & PyTorch)

While our primary implementation utilizes JAX for XLA compilation, STATIC is inherently portable to PyTorch and CUDA environments. The shift to a vectorized **Gather**-based kernel unifies the implementation strategy across frameworks. In JAX/XLA, the kernel relies on `jnp.take` (Gather). In PyTorch/Inductor, this logic translates directly to `torch.gather`.

B Analysis of STATIC Memory Usage

We present an analysis of the HBM memory usage of STATIC.

B.1 Absolute Upper Bound

The core memory usage for STATIC lies in the storage of the associated trie (which represents the restricted vocabulary) and the dense tensor mask D . For the dense tensor mask, the storage cost is simply the cost of storing $|\mathcal{V}|^d$ bits and $|\mathcal{V}|^d$ state IDs. The total storage cost for a trie in CSR format is a function of the number of unique prefix nodes at each level ℓ of the tree. For a restricted vocabulary of $|C|$ items, where each item is a Semantic ID (SID) of length L and each token has a vocabulary size $|\mathcal{V}|$, the number of unique nodes at level ℓ (N_ℓ) is strictly limited by two factors:

- (1) **Level Capacity:** The maximum possible unique sequences of length ℓ , which is $|\mathcal{V}|^\ell$.
- (2) **Total Constraints:** The total number of unique constraint items $|C|$, as no level can have more unique prefixes than there are total items in the vocabulary.

The absolute upper bound for nodes at level ℓ is therefore:

$$N_\ell \leq \min(|\mathcal{V}|^\ell, |C|)$$

The total storage upper bound in bytes, $U_{\max}(K_1, K_2, |\mathcal{V}|, |C|, L, d)$, is given by a contribution of $(\frac{1}{8} + K_2)|\mathcal{V}|^d$ for the dense tensor mask (1 bit for the mask and K_2 bytes per state, where $K_2 = 4$ typically), as well as by multiplying the sum of the per-level prefix node upper bounds for later levels by the storage cost per node K_1 (typically 12 bytes, due to the three arrays of the CSR sparse matrix format):

$$U_{\max}(K_1, K_2, |\mathcal{V}|, |C|, L, d) = \left(\frac{1}{8} + K_2\right)|\mathcal{V}|^d + K_1 \cdot \sum_{\ell=d+1}^L \min(|\mathcal{V}|^\ell, |C|)$$

B.2 Calculation for YouTube

We compute the upper bound for the YouTube setting used in Section 5. Our main configuration variables are fixed to $|\mathcal{V}| = 2048$, $L = 8$, $K_1 = 12$, $K_2 = 4$, and $d = 2$. Note since we have 20 million constraints in the restricted vocabulary, we also have $|C| = 20 \cdot 10^6$. Now we compute the total memory usage

- (1) **Dense Mask Phase** ($\ell = 1, 2$):
 - The number of bytes we need to store the dense mask is simply $(\frac{1}{8} + 4) \cdot 2048^2 = 17301504$ bytes, or ≈ 17.3 MB.
- (2) **Constraint Phase** ($\ell \geq 3$):
 - For $3 \leq \ell \leq 8$, the capacity $|\mathcal{V}|^\ell$ is significantly larger than $|C|$ (e.g., $|\mathcal{V}|^3 \approx 8.5$ billion). Hence, each of these 6 levels contributes $|C| \cdot 12$ bytes.
 - Contribution: $(L - 2) \cdot |C| \cdot 12 = 6 \cdot 20 \cdot 10^6 \cdot 12 = 1.44$ GB.

Our maximum per-chip usage is thus ≈ 1.46 GB, validating our claim of approximately 1.5 GB of HBM usage for 20 million constraints. Due to the fact that the YouTube video distribution is highly

non-uniform and exhibits significant clustering [8], the semantic ID distribution will also, which produces a significant amount of collision in the prefix space. As such, for most restrict vocabularies, the actual HBM usage will be much less than this upper bound. In practice, we note that our utilization is $\leq 75\%$ of this amount.

B.3 Capacity Planning

In our YouTube setting (Section 5), we use the rule of thumb that every 1 million constraints introduce an additional ≈ 90 MB of HBM usage. This is because the storage cost is dominated by the constraint phase described in the previous section.

As shown above, the dense mask phase (early levels) contribution remains constant (for fixed d), while each additional level in the constraint phase adds exactly $|C| \times K_1$ bytes to the storage cost.

When $|C|$ is 1 million:

- **Dense Mask Phase Contribution:** Only $(\frac{1}{8} + 4) \cdot 2048^2 = 17301504$ bytes, or ≈ 17.3 MB.
- **Constraint Phase Contribution:** For $|C| = 10^6$, we see the total contribution is $6 \cdot 10^6 \cdot 12$ bytes = 72MB.
- **Total:** 17.3 MB + 72 MB ≈ 90 MB per million items.

For the production-scale restrict corpus in Appendix B.2 of 20 million items, we saw the transition to the constraint phase happen later, resulting in an average of ≈ 73 MB per 1 million items, making 90 MB a highly reliable linear upper bound approximation for capacity planning. A similar rule-of-thumb can be computed for different values of L , $|\mathcal{V}|$, and $|C|$ using the principles above.

B.4 Memory Footprint in Implementation

While LLM parameters are typically sharded across devices in large-scale serving setups [26], the relatively compact size of the sparse transition matrix in STATIC (e.g. 1.5 GB memory for 20 million items) allows a replication strategy where the sparse transition matrix is replicated on the HBM (High-Bandwidth Memory) of every device. This eliminates the need for cross-chip communication (All-Gather or All-Reduce) during the constraint check. Each chip independently computes the validity mask for its local batch of beams, preserving the linear scalability of the serving system.

As item corpora scale toward billions, future research should explore hierarchical storing or sharding strategies to maintain this linear scalability without saturating device memory.

C Finite State Transducer (FST) Comparison

In this section we discuss why we excluded FSTs from our list of baselines in Section 5.2. While FSTs are a standard tool for grammar-constrained decoding in NLP, they are fundamentally ill-suited for the scale and hardware requirements of modern generative retrieval. In generative retrieval, each item is represented by a unique Semantic ID (SID) of length L . For a corpus of $|C|$ items, the number of states (prefix nodes) N_ℓ at each level ℓ of the search tree is given by $N_\ell \leq \min(|\mathcal{V}|^\ell, |C|)$. For our YouTube deployment with $|C| = 20 \cdot 10^6$ items, a vocabulary size $|\mathcal{V}| = 2048$, and $L = 8$, the total state count is approximately 124 million nodes.

The feasibility of on-device decoding depends on the memory footprint per state. Under a standard OpenFst VectorFst implementation using the StdArc structure, each arc requires 16 bytes to

store input/output labels, a weight, and a next-state ID. Additionally, each state incurs a container overhead (typically 24 bytes for a `std::vector` in C++). For a trie representing 20 million items with 124 million states, this implies a total memory footprint of 4.96 GB ($124 \cdot 10^6 \cdot 40$ bytes). In contrast, STATIC employs a stacked CSR layout that stores Token ID and Next Node pairs as a contiguous tensor. This requires exactly 12 bytes per node, resulting in a total footprint of 1.5 GB for 20 million items. The pointer-based FST implementation requires over 3x the memory of STATIC.

Even if an FST fits into memory, it is fundamentally hostile to accelerators for two reasons. First, FSTs rely on recursive pointer chasing, which prevents memory coalescing and nullifies hardware prefetchers. Second, standard FST traversals involve data-dependent control flow that triggers costly graph recompilations in ML compilers like XLA. Our CPU Trie baseline shows that this pointer-chasing approach adds 31.3 ms of latency per step, while STATIC adds only 0.03 ms.

D Detailed Latency Analysis

In this section, we present a fine-grained latency overhead analysis of our STATIC method in the setting of Section 5 compared to the baselines in Table 1. All numbers given represent the per-step latency throughout the full decoding (over $L = 8$ decoding steps). All values represent the additional latency (in milliseconds) incurred by the constraint enforcement logic relative to the unconstrained baseline (which performs no masking), calculated over 100 trials.

Table 4a provides the precise numbers (means and standard deviations) for Figure 2, and the $N = 2 \cdot 10^7$ row gives the means and standard deviations for Table 1. The precise numbers (means and standard deviations) for Figure 3 are given in Table 4b.

Table 4: Per-Step Latency Overhead (ms) over the Unconstrained Baseline. We report mean and standard deviation over 100 trials.

Setting	STATIC (Ours)	Hash Bitmap	PPV Approximate	PPV Exact	CPU Trie
(a) Scaling with Constraint Size $ C $ ($ \mathcal{V} = 2048$)					
$ C = 10^5$	0.023 ± 0.008	12.335 ± 0.020	0.588 ± 0.008	6.375 ± 0.268	27.070 ± 0.416
$ C = 10^6$	0.030 ± 0.008	12.339 ± 0.016	0.846 ± 0.008	15.368 ± 0.338	27.579 ± 0.353
$ C = 10^7$	0.034 ± 0.006	12.341 ± 0.026	1.486 ± 0.020	31.765 ± 2.150	28.060 ± 0.396
$ C = 2 \cdot 10^7$	0.033 ± 0.008	12.343 ± 0.020	1.564 ± 0.033	34.116 ± 2.118	31.336 ± 0.479
$ C = 10^8$	0.039 ± 0.010	12.331 ± 0.013	1.713 ± 0.038	38.691 ± 2.469	OOM
(b) Scaling with Vocabulary Size $ \mathcal{V} $ ($ C = 10^7$)					
$ \mathcal{V} = 256$	0.040 ± 0.008	1.554 ± 0.008	1.584 ± 0.035	6.004 ± 0.228	27.359 ± 0.365
$ \mathcal{V} = 512$	0.041 ± 0.008	3.094 ± 0.008	1.543 ± 0.035	10.586 ± 0.655	27.853 ± 0.294
$ \mathcal{V} = 1024$	0.038 ± 0.008	6.180 ± 0.011	1.509 ± 0.036	18.360 ± 1.313	29.239 ± 0.491
$ \mathcal{V} = 2048$	0.034 ± 0.006	12.341 ± 0.026	1.486 ± 0.020	31.765 ± 2.150	31.176 ± 0.446
$ \mathcal{V} = 4096$	0.034 ± 0.008	24.620 ± 0.033	1.620 ± 0.025	57.066 ± 3.540	29.810 ± 0.426
$ \mathcal{V} = 8192$	0.041 ± 0.008	49.191 ± 0.090	1.668 ± 0.024	107.015 ± 9.269	29.383 ± 0.741
$ \mathcal{V} = 16384$	0.044 ± 0.008	98.273 ± 0.025	1.824 ± 0.031	287.181 ± 4.321	30.885 ± 0.276
$ \mathcal{V} = 32768$	0.041 ± 0.006	196.474 ± 0.038	2.093 ± 0.029	578.738 ± 14.596	31.676 ± 0.399

E Hardware Scaling with High Branching Factor

To isolate the hardware-level scaling efficiency of the STATIC masking kernel, we design a benchmark that stresses the vectorized burst-read mechanism under extreme sparsity and high-cardinality conditions. Specifically, we evaluate the execution time of the core Vectorized Node Transition Kernel (VNTK), i.e. Algorithm 2, as the maximum branching factor increases.

For each tested max branching factor $B \in \{2^1, 2^2, \dots, 2^{18}\}$, we dynamically set the token vocabulary size such that $|\mathcal{V}| = B$. Under

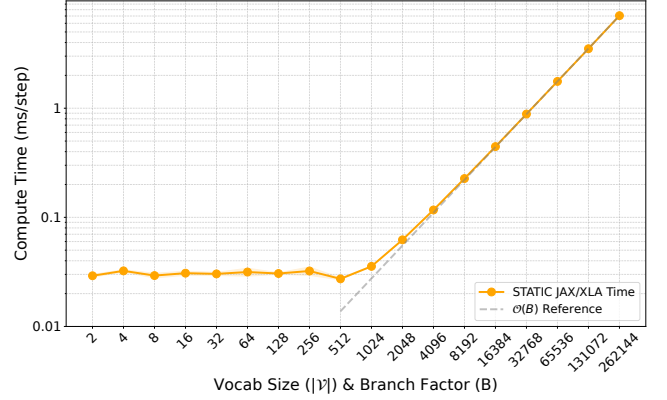


Figure 4: Scaling of the STATIC masking kernel with respect to the max branch factor (log-log scale). We plot means and standard deviations (shaded region) over 100 trials. For each trial, the token vocabulary size is set to the branch factor, while the number of constraints is fixed at $|C| = 10^6$. The STATIC method exhibits asymptotically linear $O(B)$ scaling.

this configuration, we synthesize a new constraint set comprising $|C| = 10^6$ Semantic IDs generated uniformly at random. We then flatten this newly generated prefix tree into a Compressed Sparse Row (CSR) matrix for the VNTK algorithm.

During the timed trials, we execute the JIT-compiled masking kernel, which performs a coalesced read of B contiguous elements from HBM and applies the validity mask. To ensure robust measurements, we perform synchronous wall-clock timing over multiple iterations, capturing the amortized pure hardware compute time per operation while aggressively clearing the accelerator memory between scales to prevent out-of-memory (OOM) errors. The scaling results are illustrated in Figure 4. Crucially, the benchmark demonstrates that STATIC exhibits strict asymptotic linear scaling, $O(B)$, with respect to the maximum branch factor B . We note that STATIC exhibits constant runtime before reaching TPU VMEM bandwidth, at which point the linear regime begins.

F Cold-Start Experiment Details

Here, we add additional details of the evaluation protocol and dense retrieval baseline for the cold-start experiments from Section 6.

The evaluation protocol for the Amazon dataset is as follows. For each item history sequence in the cold-start split, the model generates output items either through autoregressive decoding of Semantic IDs or by dense retrieval. We report the peak Recall@1 results achieved over 70 epochs of training, which we found sufficient for convergence across the tested subdatasets.

The dense retrieval baseline is a two-tower model designed to align sequential user behavior with static item content features. The “query tower” utilizes the Gemma 1B architecture to convert the sequence of Semantic IDs into a 128-dimensional query vector using a projection head. The “item tower” is a Multi-Layer Perceptron (MLP) with hidden layer dimensions of (512, 256) and interspersed dropout layers that have a dropout rate of 0.1. This tower ingests precomputed 768-dimensional T5 item embeddings [25] to obtain

an 128-dimensional item representation. The dense model is optimized using an in-batch contrastive loss with a batch size of 512 to maximize the similarity between query and ground-truth item vectors. The constrained recommendation output is generated by Maximum Inner Product Search (MIPS) with only the cold-start item vectors, for a fair comparison with other cold-start techniques.

G STATIC Code: JAX Implementation

In this section, we give a code snippet to demonstrate an explicit example implementation of Algorithms 1 and 2 using the JAX [2] and Flax [10] frameworks. Note that the code works on both TPU and GPU with almost identical latency overhead.

```
# Copyright 2026 Google LLC.
# SPDX-License-Identifier: Apache-2.0
"""Implementation of Hardware-Accelerated Constrained Decoding algorithms."""

import functools
import flax
import jax
import jax.numpy as jnp

NEG_INF = -1.0e10

@flax.struct.dataclass(frozen=True)
class TransitionMatrix:
    """CSR-based Transition Matrix with optional dense optimizations."""

    row_pointers: jnp.ndarray
    data: jnp.ndarray
    start_mask_packed: jnp.ndarray | None = None
    l1_dense_mask_packed: jnp.ndarray | None = None
    l1_dense_states: jnp.ndarray | None = None

def vectorized_sparse_candidate_extraction(
    log_probs, nodes, transition_matrix, layer_max_branches, vocab_size
):
    """Algorithm 2: Vectorized Node Transition Kernel (VNTK)."""

    n_flat, lp_flat = nodes.reshape(-1), log_probs.reshape(-1, vocab_size)
    starts = transition_matrix.row_pointers[n_flat]
    lens = transition_matrix.row_pointers[n_flat + 1] - starts
    offsets = jnp.arange(layer_max_branches)
    gathered = jnp.take(
        transition_matrix.data,
        starts[:, None] + offsets[None, :],
        axis=0,
        mode="fill",
        fill_value=0,
    )
    valid_mask = offsets[None, :] < lens[:, None]
    dense_indices, dense_data = gathered[:, :, 0], gathered[:, :, 1]
    dense_data = jnp.where(
        valid_mask, dense_data, 0
    )
    candidate_lp = jnp.take_along_axis(
        lp_flat, jnp.clip(dense_indices, max=vocab_size - 1), axis=1
    )
    return (
        jnp.where(valid_mask, candidate_lp, NEG_INF),
        dense_indices,
        dense_data,
        valid_mask,
    )

@functools.partial(
    jax.jit, static_argnames=("vocab_size", "layer_max_branches", "step")
)
def hardware_accelerated_constrained_decoding_step(
    logits,
    previous_nodes,
    transition_matrix,
    layer_max_branches,
    vocab_size,
    step=-1,
):
    """Algorithm 1: Hardware-Accelerated Constrained Decoding Step."""
    log_probs = jax.nn.log_softmax(logits)

    if step == 0 and transition_matrix.start_mask_packed is not None:
        mask = jnp.unpackbits(transition_matrix.start_mask_packed).astype(bool)[
            :vocab_size
        ]
        return jnp.where(mask, log_probs, NEG_INF), jnp.broadcast_to(
            jnp.arange(vocab_size, dtype=jnp.int32) + 1, logits.shape
        )
    if step == 1 and transition_matrix.l1_dense_mask_packed is not None:
        parents = previous_nodes - 1
        mask = jnp.unpackbits(
            transition_matrix.l1_dense_mask_packed[parents], axis=-1
        )
        return (
            jnp.where(mask.astype(bool)[..., :vocab_size], log_probs, NEG_INF),
            transition_matrix.l1_dense_states[parents],
        )
    sparse_lp, sparse_toks, sparse_next, valid_mask = (
        vectorized_sparse_candidate_extraction(
            log_probs,
            previous_nodes,
            transition_matrix,
            layer_max_branches,
            vocab_size,
        )
    )
    scatter_idx = jnp.where(valid_mask, sparse_toks, vocab_size)
    masked_lp = jnp.full((previous_nodes.size, vocab_size + 1), NEG_INF)
    masked_lp = masked_lp.at[
        jnp.arange(previous_nodes.size)[:, None], scatter_idx
    ].set(sparse_lp)

    return (
        masked_lp[:, :vocab_size].reshape(logits.shape),
        sparse_next.reshape(previous_nodes.shape + (layer_max_branches,)),
    )
```
